Relatório Técnico — PMW Tasks

Disciplina Programação e Modelagem para Web (PMW) — FSG · Prof. Adelcio Biazi · João Alfredo Tamagno · Giovani Rigotti

1. Estrutura do projeto

O PMW Tasks é um gerenciador de tarefas em HTML, CSS e JavaScript puro (sem frameworks nem build), com dados persistidos em `localStorage`. O fluxo é: o usuário se **registra**, faz **login** e acessa três áreas — **Home** (lista de tarefas), **Criar Tarefa** e **Perfil** (troca de senha). Cada usuário vê apenas as próprias tarefas, e o status de cada uma é *Pendente*, *Atrasada* ou *Concluída*. Os arquivos JS são separados por responsabilidade, e o layout comum é reaproveitado entre as páginas por Web Components:

```
PWMTasks/
├── index.html          Login
├── registro.html       Cadastro de usuário
├── home.html           Lista de tarefas do usuário logado (concluir/excluir)
├── createTask.html     Criação de tarefa (data não pode ser passada)
├── perfil.html         Dados do usuário + troca de senha
├── css/style.css       Design system (cores, raio, sombra), layout e responsivo
└── js/
    ├── storage.js      Único ponto de acesso ao localStorage
    ├── layout.js       Web Components do cabeçalho/menu e do rodapé
    ├── auth.js         Login, registro, logout e guarda de sessão
    ├── tasks.js        Listagem, criação, conclusão e exclusão de tarefas
    └── profile.js      Perfil e troca de senha
```

A descrição detalhada de cada área e o resumo das funções de código estão no `README.md`.

2. Principais decisões técnicas

- **Modularização por responsabilidade:** cada arquivo JS cuida de uma parte (dados, autenticação, tarefas, perfil, layout), facilitando manutenção e leitura.
- **`storage.js` como única porta de entrada ao `localStorage`:** nenhum outro arquivo o acessa diretamente, o que centraliza as chaves (`usuarios`, `tarefas`, `userLogado`).
- **Web Components nativos (`customElements`)** para cabeçalho/menu e rodapé: cada página só declara `<site-header>`/`<site-footer>`. Escolhidos em vez de `fetch` de HTML parcial para evitar requisições extras e manter o layout em um único lugar.
- **Design system em variáveis CSS** (cores, raios, sombras) aplicado a botões, cards, inputs e badges de urgência/status. O cabeçalho e o rodapé usam um tom de azul inspirado na identidade da FSG, para dar destaque sem exagero.
- **Responsividade orientada a conteúdo:** abaixo de 600px a tabela vira uma lista de cards (em vez de rolagem lateral) e o rodapé é simplificado; o cabeçalho usa `position: sticky`.
- **Status "Atrasada" calculado, não salvo:** a cada exibição, tarefa não concluída com data anterior a hoje aparece como atrasada, sem gravar nada a mais.
- **Validação de data em duas camadas:** o campo recebe `min` com a data de hoje e o envio confere de novo (a data pode ser digitada). Hoje é montado com `getFullYear/getMonth/getDate` (fuso local), pois `toISOString()` usa UTC e erraria o dia à noite no Brasil.

3. Demonstração de como cada requisito foi atendido

Requisito	Onde	Como
Manipulação de objetos HTML e eventos com DOM	`auth.js`, `tasks.js`, `profile.js`	Listeners de `submit` nos formulários de login, registro, tarefa e senha; clique nas linhas da tabela e no botão "Excluir"; modal de confirmação criado e removido dinamicamente.
Uso de vetores/matrizes	`storage.js`, `tasks.js`	Usuários e tarefas são arrays manipulados com `filter`, `find`/`findIndex`, `map` e `forEach` (ex.: filtrar as tarefas do usuário logado, localizar a tarefa clicada).

Criação de objetos em tempo de execução	`tasks.js`, `layout.js`	`document.createElement` gera as linhas da tabela e o modal; os objetos de tarefa/usuário recebem propriedades (`id`, `done`, `user`) ao serem criados; os Web Components montam cabeçalho e rodapé na execução.
CRUD completo com localStorage	`storage.js` + `auth.js`, `tasks.js`, `profile.js`	**Create:** cadastro de usuário e criação de tarefa. **Read:** login e listagem. **Update:** concluir tarefa e trocar senha. **Delete:** excluir tarefa, com confirmação em modal.
Design responsivo e amigável com CSS	`style.css`	`@media (max-width: 600px)` transforma a tabela em cards e compacta o rodapé; cabeçalho fixo; estados de foco e hover; badges coloridos por urgência e status (incluindo Atrasada em vermelho).

4. Como utilizar

1. Na tela de login, clique em **Registre-se**, informe nome e senha e volte ao login para entrar: você chega à **Home**.
2. No menu, clique em **Criar Tarefa**, preencha nome, descrição, data prevista (hoje ou futura) e urgência, e confirme. A tarefa aparece na Home como **Pendente**.
3. Para concluir, clique na linha da tarefa e confirme: o status vira **Concluída**. Para apagar, use o botão **Excluir** e confirme.
4. Se o prazo passar sem a tarefa ser concluída, ela aparece como **Atrasada**, em vermelho.
5. Em **Perfil**, altere a senha informando a atual, a nova e a confirmação. O botão **Sair** encerra a sessão.

Os dados ficam no navegador usado; limpar os dados do site ou trocar de navegador/computador faz contas e tarefas não aparecerem.

Repositórios: github.com/Audamon/PWMTasks · github.com/GRigotti/PWMTasks